

Pengolahan Citra Digital

Bab 4: Pemrosesan Citra dalam Domain Frekuensi

Memahami Transformasi Fourier, teorema sampling, dan teknik pemfilteran berbasis frekuensi sebagai pendekatan alternatif yang powerful dalam pengolahan citra.

Contents

1	Pendahuluan	2
2	Intuisi Dasar: Ide Besar Fourier (Bagian 4.1 & 4.2)	2
2.1	Teorema Fourier	2
2.2	Pasangan Transformasi Fourier 1-D Kontinu	2
2.3	Representasi Kompleks: Spektrum, Fase, dan Daya	3
2.4	Sifat-Sifat Penting Transformasi Fourier	3
3	Sampling dan Teorema Sampling (Bagian 4.3)	3
3.1	Kebutuhan Sampling	3
3.2	Teorema Sampling Nyquist-Shannon	3
3.3	Aliasing	4
3.4	Rekonstruksi Sinyal: Interpolasi Sinc	4
4	Discrete Fourier Transform 2-D (Bagian 4.5 & 4.6)	4
4.1	DFT 1-D Diskrit	4
4.2	DFT 2-D untuk Citra $M \times N$	5
4.3	Spektrum Amplitudo 2-D	5
4.4	Sudut Fase 2-D	5
4.5	Periodisitas dan Centering	5
4.6	Fast Fourier Transform (FFT)	5
5	Alur Kerja Pemfilteran Domain Frekuensi (Bagian 4.7)	6
5.1	Teorema Konvolusi sebagai Landasan	6
5.2	Langkah-Langkah Lengkap Pemfilteran	6
5.3	Contoh Kode: Alur Kerja Lengkap FFT Filtering	7
6	Jenis-Jenis Filter (Bagian 4.8 & 4.9)	8
6.1	Jarak dari Pusat Spektrum	8
6.2	Hubungan Lowpass dan Highpass	8

6.3	Filter Lowpass (Smoothing)	8
6.3.1	Ideal Lowpass Filter (ILPF)	8
6.3.2	Butterworth Lowpass Filter (BLPF)	9
6.3.3	Gaussian Lowpass Filter (GLPF)	9
6.4	Filter Highpass (Sharpening)	9
6.5	High-Frequency Emphasis (HFE) Filter	9
6.6	Contoh Kode: Perbandingan Filter Lowpass dan Visualisasi Spektrum	10
7	Homomorphic Filtering (Bagian 4.9)	11
7.1	Model Pencahayaan-Reflektansi	11
7.2	Solusi: Linearisasi dengan Logaritma	11
7.3	Filter Homomorphic	12
7.4	Alur Kerja Lengkap Homomorphic Filtering	12
7.5	Contoh Kode: Homomorphic Filtering	12
8	Ringkasan Perbandingan Filter Frekuensi	14
9	Mini-Latihan (Active Learning)	14

1 Pendahuluan

Pada Bab 3 kita memproses citra langsung pada nilai piksel (*domain spasial*). Bab 4 memperkenalkan pendekatan yang sama sekali berbeda: memindahkan citra ke **domain frekuensi**, melakukan operasi di sana, lalu memindahkannya kembali.

Analogi yang tepat: bayangkan sebuah lagu. Kita bisa mendengarnya sebagai gelombang tekanan udara yang berubah terhadap waktu (*domain waktu*), atau kita bisa melihat *lembar musiknya* yang menunjukkan nada dan amplitudo setiap frekuensi (*domain frekuensi*). Keduanya merepresentasikan informasi yang *sama*, hanya caranya berbeda.

Dalam konteks citra:

- **Frekuensi rendah** merepresentasikan variasi lambat – latar belakang, gradien halus, dan area dengan intensitas seragam.
- **Frekuensi tinggi** merepresentasikan variasi cepat – tepi (*edges*), noise, dan detail halus.

Keunggulan domain frekuensi: konvolusi dengan kernel besar yang mahal di domain spasial menjadi sekadar perkalian elemen-per-elemen di domain frekuensi, jauh lebih efisien.

2 Intuisi Dasar: Ide Besar Fourier (Bagian 4.1 & 4.2)

2.1 Teorema Fourier

Jean Baptiste Joseph Fourier (1807) mengemukakan ide yang pada masanya dianggap radikal: *setiap fungsi periodik dapat direpresentasikan sebagai penjumlahan tak terhingga dari sines dan cosines dengan frekuensi berbeda*, masing-masing dikalikan dengan koefisien tertentu. Ide ini kemudian digeneralisasikan untuk fungsi non-periodik melalui **Transformasi Fourier**.

2.2 Pasangan Transformasi Fourier 1-D Kontinu

Untuk fungsi kontinu $f(t)$, **Transformasi Fourier Maju** didefinisikan sebagai:

$$F(\mu) = \int_{-\infty}^{\infty} f(t) e^{-j2\pi\mu t} dt \quad (1)$$

Dan **Transformasi Fourier Balik** (Inverse FT):

$$f(t) = \int_{-\infty}^{\infty} F(\mu) e^{j2\pi\mu t} d\mu \quad (2)$$

di mana μ mewakili frekuensi (dalam Hz jika t dalam detik) dan $j = \sqrt{-1}$. Berdasarkan rumus Euler: $e^{j\theta} = \cos \theta + j \sin \theta$, sehingga setiap komponen Fourier pada dasarnya adalah gelombang sinus/cosinus.

Intuisi Persamaan (1): Faktor $e^{-j2\pi\mu t}$ berfungsi sebagai “detektor” frekuensi. Mengalikan $f(t)$ dengan detektor frekuensi μ lalu mengintegrasikan akan mengekstrak seberapa kuat komponen frekuensi μ terdapat di dalam $f(t)$.

2.3 Representasi Kompleks: Spektrum, Fase, dan Daya

Karena $F(\mu)$ adalah bilangan kompleks, ia dapat ditulis sebagai:

$$F(\mu) = R(\mu) + j I(\mu) \quad (3)$$

Spektrum Amplitudo (yang umum divisualisasikan):

$$|F(\mu)| = \sqrt{R^2(\mu) + I^2(\mu)} \quad (4)$$

Sudut Fase (menyimpan informasi lokasi dan struktur):

$$\phi(\mu) = \arctan\left(\frac{I(\mu)}{R(\mu)}\right) \quad (5)$$

Spektrum Daya (*Power Spectrum*):

$$P(\mu) = |F(\mu)|^2 = R^2(\mu) + I^2(\mu) \quad (6)$$

2.4 Sifat-Sifat Penting Transformasi Fourier

Sifat	Domain Spasial/Waktu	Domain Frekuensi
Linearitas	$af(t) + bg(t)$	$aF(\mu) + bG(\mu)$
Pergeseran	$f(t - t_0)$	$e^{-j2\pi\mu t_0} F(\mu)$
Konvolusi	$f(t) * h(t)$	$F(\mu) \cdot H(\mu)$
Korelasi	$f(t) \star g(t)$	$F^*(\mu) \cdot G(\mu)$
Simetri	$f(t)$ riil	$F(-\mu) = F^*(\mu)$

Sifat Konvolusi adalah kunci utama bab ini: konvolusi di domain spasial setara dengan perkalian di domain frekuensi (lihat Persamaan (17)). Ini mengubah operasi yang mahal menjadi perkalian sederhana setelah FFT.

3 Sampling dan Teorema Sampling (Bagian 4.3)

3.1 Kebutuhan Sampling

Sebelum citra dapat diolah komputer, sinyal kontinu harus di-*sampling* – diambil nilainya pada titik-titik diskrit berjarak ΔT . Pertanyaan kritis: **seberapa rapat sampling agar tidak ada informasi yang hilang?**

3.2 Teorema Sampling Nyquist-Shannon

Sebuah fungsi yang dibatasi pita frekuensinya (*band-limited*) dengan frekuensi tertinggi $\mu_{\max}$ dapat dipulihkan *secara sempurna* dari sampel-sampelnya jika dan hanya jika laju sampling memenuhi:

$$\frac{1}{\Delta T} > 2\mu_{\max} \quad (7)$$

Nilai $2\mu_{\max}$ disebut **Laju Nyquist**. Interval sampling $\Delta T = \frac{1}{2\mu_{\max}}$ disebut **Interval Nyquist**.

3.3 Aliasing

Jika Persamaan (7) dilanggar (sampling terlalu jarang), terjadi **aliasing**: komponen frekuensi tinggi “berpura-pura” menjadi frekuensi rendah yang salah. Secara matematis, spektrum sinyal yang di-sampling bersifat periodik, dan jika laju sampling tidak cukup, periode-periode tersebut akan *saling tumpang tindih (spectral overlap)*.

Lihat ilustrasi visual aliasing pada gelombang sinus di [FIGURE 4.11].

Contoh Aliasing di Dunia Nyata:

- Roda berputar di film yang tampak berputar mundur (*wagon-wheel effect*).
- Garis diagonal pada layar yang tampak bergerigi (*jaggies*).
- Pola Moire pada foto kain bermotif halus.

Solusi umum: terapkan *anti-aliasing filter* (lowpass) sebelum sampling.

3.4 Rekonstruksi Sinyal: Interpolasi Sinc

Sinyal kontinu dapat direkonstruksi sempurna dari sampelnya menggunakan:

$$f(t) = \sum_{n=-\infty}^{\infty} f(n\Delta T) \operatorname{sinc}\left(\frac{t - n\Delta T}{\Delta T}\right) \quad (8)$$

di mana $\operatorname{sinc}(x) = \frac{\sin(\pi x)}{\pi x}$.

4 Discrete Fourier Transform 2-D (Bagian 4.5 & 4.6)

4.1 DFT 1-D Diskrit

Untuk sinyal diskrit $f(x)$ dengan N sampel, DFT 1-D didefinisikan sebagai:

$$F(u) = \sum_{x=0}^{N-1} f(x) e^{-j2\pi ux/N}, \quad u = 0, 1, \dots, N-1 \quad (9)$$

Dan inversnya (IDFT 1-D):

$$f(x) = \frac{1}{N} \sum_{u=0}^{N-1} F(u) e^{j2\pi ux/N}, \quad x = 0, 1, \dots, N-1 \quad (10)$$

4.2 DFT 2-D untuk Citra $M \times N$

$$F(u, v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x, y) e^{-j2\pi(\frac{ux}{M} + \frac{vy}{N})} \quad (11)$$

Dan inversnya (IDFT 2-D):

$$f(x, y) = \frac{1}{MN} \sum_{u=0}^{M-1} \sum_{v=0}^{N-1} F(u, v) e^{j2\pi(\frac{ux}{M} + \frac{vy}{N})} \quad (12)$$

4.3 Spektrum Amplitudo 2-D

Karena $F(u, v)$ adalah bilangan kompleks, spektrum amplitudo yang divisualisasikan:

$$|F(u, v)| = \sqrt{R^2(u, v) + I^2(u, v)} \quad (13)$$

Karena rentang nilainya sangat besar, biasanya ditampilkan dalam skala logaritmik:

$$D(u, v) = c \cdot \log(1 + |F(u, v)|) \quad (14)$$

4.4 Sudut Fase 2-D

$$\phi(u, v) = \arctan\left(\frac{I(u, v)}{R(u, v)}\right) \quad (15)$$

Fase vs. Amplitudo: Jika kita merekonstruksi citra hanya menggunakan informasi fase (amplitudo diset konstan), bentuk objek masih dapat dikenali. Sebaliknya, jika hanya menggunakan amplitudo (fase diset nol), citra menjadi kacau. Ini menunjukkan bahwa **fase menyimpan informasi struktural** citra. Lihat [FIGURE 4.26].

4.5 Periodisitas dan Centering

DFT bersifat periodik dengan periode M dan N . Akibatnya, frekuensi rendah (komponen DC) berada di sudut-sudut spektrum, bukan di tengah. Untuk memudahkan visualisasi dan pemfilteran, spektrum digeser agar frekuensi rendah berada di tengah dengan mengalikan citra sebelum DFT dengan:

$$f_c(x, y) = f(x, y) \cdot (-1)^{x+y} \quad (16)$$

Setelah pemrosesan selesai, kalikan kembali dengan $(-1)^{x+y}$ untuk mengembalikan ke posisi semula.

4.6 Fast Fourier Transform (FFT)

DFT naif memerlukan $O(N^2)$ operasi untuk sinyal 1-D berukuran N . **FFT** (algoritma Cooley-Tukey, 1965) mereduksinya menjadi $O(N \log N)$. Untuk citra $N \times N$, DFT 2-D naif memerlukan $O(N^4)$, sementara FFT 2-D hanya $O(N^2 \log N)$ – perbedaan yang sangat signifikan pada citra besar.

Ukuran Citra	DFT Naif (operasi)	FFT (operasi)
256×256	$\approx 4 \times 10^9$	$\approx 1 \times 10^6$
512×512	$\approx 7 \times 10^{10}$	$\approx 4 \times 10^6$
1024×1024	$\approx 1 \times 10^{12}$	$\approx 2 \times 10^7$

5 Alur Kerja Pemfilteran Domain Frekuensi (Bagian 4.7)

5.1 Teorema Konvolusi sebagai Landasan

Sifat konvolusi dari Transformasi Fourier menyatakan:

$$f(x, y) * h(x, y) \xleftrightarrow{\mathcal{F}} F(u, v) \cdot H(u, v) \quad (17)$$

Persamaan (17) adalah motivasi utama pemfilteran domain frekuensi: konvolusi spasial yang kompleks diganti dengan perkalian sederhana setelah FFT.

5.2 Langkah-Langkah Lengkap Pemfilteran

Diberikan citra masukan $f(x, y)$ berukuran $M \times N$:

1. **Padding.** Perluas citra menjadi $P \times Q$ dengan menambah nol (*zero-padding*):

$$P \geq 2M - 1, \quad Q \geq 2N - 1 \quad (18)$$

Biasanya dibulatkan ke pangkat 2 terdekat untuk efisiensi FFT.

2. **Centering.** Kalikan dengan $(-1)^{x+y}$ (Persamaan (16)).

3. **DFT.** Hitung:

$$F_p(u, v) = \mathcal{F}\{f_p(x, y)\} \quad (19)$$

4. **Filter.** Kalikan dengan fungsi filter:

$$G(u, v) = H(u, v) \cdot F_p(u, v) \quad (20)$$

5. **IDFT.** Hitung:

$$g_p(x, y) = \mathcal{F}^{-1}\{G(u, v)\} \quad (21)$$

6. **Post-processing.** Ambil bagian riil, kembalikan centering, dan potong ke ukuran asli:

$$g(x, y) = \text{Re}[g_p(x, y) \cdot (-1)^{x+y}] \Big|_{\substack{0 \leq x \leq M-1 \\ 0 \leq y \leq N-1}} \quad (22)$$

Mengapa Perlu Padding? Tanpa padding, DFT menghasilkan *circular convolution* (konvolusi siklik): piksel dari sisi kanan “bocor” ke sisi kiri. Padding memastikan konvolusi siklik bersifat identik dengan konvolusi linear pada region yang kita inginkan, menghindari *wraparound error*.

5.3 Contoh Kode: Alur Kerja Lengkap FFT Filtering

Listing 1: Alur kerja pemfilteran domain frekuensi (6 langkah)

```

1 import numpy as np
2 from PIL import Image
3 import matplotlib.pyplot as plt
4
5 def freq_domain_filter(img, H):
6     """
7     Terapkan filter H(u,v) ke citra menggunakan domain frekuensi.
8     img : numpy array 2D (grayscale)
9     H    : numpy array 2D, ukuran P x Q (setelah padding)
10    """
11    M, N = img.shape
12    P, Q = H.shape
13
14    # Langkah 1: Padding
15    fp = np.zeros((P, Q))
16    fp[:M, :N] = img
17
18    # Langkah 2: Centering -- (-1)^(x+y)
19    x, y = np.arange(P), np.arange(Q)
20    xx, yy = np.meshgrid(x, y, indexing='ij')
21    center_mask = (-1.0) ** (xx + yy)
22    fp_c = fp * center_mask
23
24    # Langkah 3: DFT
25    F = np.fft.fft2(fp_c)
26
27    # Langkah 4: Filter G(u,v) = H(u,v) * F(u,v)
28    G = H * F
29
30    # Langkah 5: IDFT
31    gp = np.fft.ifft2(G)
32
33    # Langkah 6: Post-processing
34    g = np.real(gp) * center_mask # balik centering
35    g = g[:M, :N]                # potong ke ukuran asli
36
37    return np.clip(g, 0, 255)
38
39
40 # --- Demo dengan Ideal Lowpass Filter ---
41 img = np.array(Image.open("image.jpg").convert("L"), dtype=np.float64
42 )
43 M, N = img.shape
44 P, Q = 2 * M, 2 * N
45 # Buat filter Ideal Lowpass (centered)

```

```

46 D0 = 50
47 u = np.fft.fftfreq(P) * P
48 v = np.fft.fftfreq(Q) * Q
49 uu, vv = np.meshgrid(u, v, indexing='ij')
50 D = np.sqrt(uu**2 + vv**2)
51 H_ilpf = (D <= D0).astype(np.float64)
52
53 result = freq_domain_filter(img, H_ilpf)
54
55 fig, axes = plt.subplots(1, 2, figsize=(10, 5))
56 axes[0].imshow(img.astype(np.uint8), cmap='gray')
57 axes[0].set_title('Citra Asli')
58 axes[1].imshow(result.astype(np.uint8), cmap='gray')
59 axes[1].set_title('Setelah ILPF (D0=50)')
60 for ax in axes:
61     ax.axis('off')
62 plt.tight_layout()
63 plt.show()

```

6 Jenis-Jenis Filter (Bagian 4.8 & 4.9)

6.1 Jarak dari Pusat Spektrum

Dasar dari semua filter frekuensi radial adalah jarak setiap titik (u, v) dari pusat (DC component):

$$D(u, v) = \sqrt{u^2 + v^2} \quad (23)$$

Seluruh filter lowpass dan highpass isotropik didefinisikan berdasarkan Persamaan (23) dan nilai cutoff D_0 .

6.2 Hubungan Lowpass dan Highpass

Setiap filter highpass dapat diturunkan langsung dari filter lowpass:

$$H_{HP}(u, v) = 1 - H_{LP}(u, v) \quad (24)$$

6.3 Filter Lowpass (Smoothing)

6.3.1 Ideal Lowpass Filter (ILPF)

$$H_{ILPF}(u, v) = \begin{cases} 1 & \text{jika } D(u, v) \leq D_0 \\ 0 & \text{jika } D(u, v) > D_0 \end{cases} \quad (25)$$

Transisi yang sangat tajam di domain frekuensi (Persamaan (25)) menghasilkan osilasi (*ringing / Gibbs phenomenon*) di domain spasial – berupa lingkaran-lingkaran di sekitar tepi objek.

Lihat efek *ringing* dari ILPF pada [FIGURE 4.41].

6.3.2 Butterworth Lowpass Filter (BLPF)

$$H_{\text{BLPF}}(u, v) = \frac{1}{1 + \left[\frac{D(u, v)}{D_0} \right]^{2n}} \quad (26)$$

di mana n adalah orde filter. Semakin besar n , semakin tajam transisi dan semakin mirip ILPF, namun ringing juga semakin muncul.

Panduan Pemilihan Orde: $n = 1$ tidak menghasilkan ringing; $n = 2$ merupakan kompromi yang baik; $n \geq 5$ mulai menunjukkan ringing.

6.3.3 Gaussian Lowpass Filter (GLPF)

$$H_{\text{GLPF}}(u, v) = e^{-D^2(u, v) / (2D_0^2)} \quad (27)$$

Keunggulan unik: Transformasi Fourier dari fungsi Gaussian adalah Gaussian itu sendiri. Ini berarti GLPF di domain frekuensi berkorespondensi dengan Gaussian filter di domain spasial, dan Gaussian **tidak menghasilkan ringing sama sekali**.

Perbandingan ketiga filter lowpass dapat dilihat pada [FIGURE 4.44].

6.4 Filter Highpass (Sharpening)

Menggunakan Persamaan (24), ketiga filter highpass:

$$H_{\text{IHPF}}(u, v) = \begin{cases} 0 & \text{jika } D(u, v) \leq D_0 \\ 1 & \text{jika } D(u, v) > D_0 \end{cases} \quad (28)$$

$$H_{\text{BHPF}}(u, v) = \frac{1}{1 + \left[\frac{D_0}{D(u, v)} \right]^{2n}} \quad (29)$$

$$H_{\text{GHPF}}(u, v) = 1 - e^{-D^2(u, v) / (2D_0^2)} \quad (30)$$

6.5 High-Frequency Emphasis (HFE) Filter

Filter highpass murni menghilangkan seluruh komponen DC sehingga citra tampak “datar” tanpa tonalitas. **HFE Filter** mengatasi ini dengan mempertahankan sebagian komponen frekuensi rendah:

$$H_{\text{HFE}}(u, v) = a + b \cdot H_{\text{HP}}(u, v), \quad a \geq 0, \quad b > a \quad (31)$$

di mana a adalah offset (biasanya $0 < a < 1$) dan b adalah faktor penguatan frekuensi tinggi.

Jika dikombinasikan dengan Histogram Equalization setelah filtering, hasilnya sangat efektif untuk meningkatkan ketajaman dan kontras secara bersamaan. Lihat [FIGURE 4.57].

6.6 Contoh Kode: Perbandingan Filter Lowpass dan Visualisasi Spektrum

Listing 2: Perbandingan ILPF, BLPF, GLPF dan visualisasi spektrum

```

1 import numpy as np
2 from PIL import Image
3 import matplotlib.pyplot as plt
4
5 def make_distance_matrix(P, Q):
6     u = np.fft.fftfreq(P) * P
7     v = np.fft.fftfreq(Q) * Q
8     uu, vv = np.meshgrid(u, v, indexing='ij')
9     D = np.sqrt(uu**2 + vv**2)
10    D[D == 0] = 1e-10
11    return D
12
13 def make_lowpass_filters(P, Q, D0, n=2):
14     D = make_distance_matrix(P, Q)
15     H_ideal = (D <= D0).astype(float)
16     H_butter = 1.0 / (1.0 + (D / D0) ** (2 * n))
17     H_gauss = np.exp(-(D**2) / (2 * D0**2))
18     return H_ideal, H_butter, H_gauss
19
20 def apply_filter(img, H):
21     M, N = img.shape
22     P, Q = H.shape
23     fp = np.zeros((P, Q))
24     fp[:M, :N] = img
25     x, y = np.arange(P), np.arange(Q)
26     xx, yy = np.meshgrid(x, y, indexing='ij')
27     mask = (-1.0) ** (xx + yy)
28     F = np.fft.fft2(fp * mask)
29     G = H * F
30     g = np.real(np.fft.ifft2(G)) * mask
31     return np.clip(g[:M, :N], 0, 255)
32
33 img = np.array(Image.open("image.jpg").convert("L"), dtype=np.float64)
34 M, N = img.shape
35 P, Q = 2 * M, 2 * N
36 D0 = 30
37
38 # Spektrum Fourier
39 x, y = np.arange(M), np.arange(N)
40 xx, yy = np.meshgrid(x, y, indexing='ij')
41 mask = (-1.0) ** (xx + yy)
42 F = np.fft.fft2(img * mask)
43 spectrum = np.log(1 + np.abs(F))
44 spectrum = 255 * spectrum / spectrum.max()
45
46 # Filter

```

```

47 H_ideal, H_butter, H_gauss = make_lowpass_filters(P, Q, D0, n=2)
48
49 fig, axes = plt.subplots(2, 3, figsize=(15, 8))
50
51 axes[0, 0].imshow(img.astype(np.uint8), cmap='gray')
52 axes[0, 0].set_title('Citra Asli')
53 axes[0, 1].imshow(spectrum.astype(np.uint8), cmap='gray')
54 axes[0, 1].set_title('Spektrum Fourier (log)')
55 axes[0, 2].axis('off')
56
57 axes[1, 0].imshow(apply_filter(img, H_ideal).astype(np.uint8), cmap='
    gray')
58 axes[1, 0].set_title('ILPF (D0=30)')
59 axes[1, 1].imshow(apply_filter(img, H_butter).astype(np.uint8), cmap=
    'gray')
60 axes[1, 1].set_title('BLPF n=2 (D0=30)')
61 axes[1, 2].imshow(apply_filter(img, H_gauss).astype(np.uint8), cmap='
    gray')
62 axes[1, 2].set_title('GLPF (D0=30)')
63
64 for ax in axes.flatten():
65     ax.axis('off')
66 plt.tight_layout()
67 plt.show()

```

7 Homomorphic Filtering (Bagian 4.9)

7.1 Model Pencahayaan-Reflektansi

Sebuah citra $f(x, y)$ dimodelkan sebagai perkalian dua komponen:

$$f(x, y) = i(x, y) \cdot r(x, y) \quad (32)$$

di mana:

- $i(x, y)$ adalah **iluminasi** – seberapa banyak cahaya mengenai objek. Bervariasi *lambat* secara spasial $\Rightarrow$ frekuensi rendah. Rentang: $0 < i(x, y) < \infty$.
- $r(x, y)$ adalah **reflektansi** – seberapa banyak cahaya dipantulkan objek. Bervariasi *cepat* $\Rightarrow$ frekuensi tinggi. Rentang: $0 < r(x, y) < 1$.

Masalah utama: Transformasi Fourier dari perkalian bukan perkalian dari Transformasi Fourier:

$$\mathcal{F}\{i \cdot r\} \neq \mathcal{F}\{i\} \cdot \mathcal{F}\{r\}$$

Sehingga kita tidak bisa memisahkan i dan r secara langsung di domain frekuensi.

7.2 Solusi: Linearisasi dengan Logaritma

Dengan mengambil logaritma natural, perkalian berubah menjadi penjumlahan:

$$z(x, y) = \ln f(x, y) = \ln i(x, y) + \ln r(x, y) \quad (33)$$

Karena penjumlahan bersifat linear terhadap Transformasi Fourier:

$$Z(u, v) = \mathcal{F}_i(u, v) + \mathcal{F}_r(u, v) \quad (34)$$

di mana $\mathcal{F}_i$ dan $\mathcal{F}_r$ adalah DFT dari $\ln i$ dan $\ln r$.

7.3 Filter Homomorphic

Filter yang digunakan berfungsi menekan frekuensi rendah (iluminasi) dan meningkatkan frekuensi tinggi (reflektansi):

$$H(u, v) = (\gamma_H - \gamma_L) \left[1 - e^{-c D^2(u, v)/D_0^2} \right] + \gamma_L \quad (35)$$

di mana $\gamma_L < 1 < \gamma_H$, dan c mengontrol kecuraman transisi.

7.4 Alur Kerja Lengkap Homomorphic Filtering

1. Hitung logaritma: $z(x, y) = \ln[f(x, y)]$ (Persamaan (33)).
2. Hitung DFT: $Z(u, v) = \mathcal{F}\{z(x, y)\}$.
3. Terapkan filter:

$$S(u, v) = H(u, v) \cdot Z(u, v) \quad (36)$$

4. Hitung IDFT: $s(x, y) = \mathcal{F}^{-1}\{S(u, v)\}$.
5. Kembalikan ke domain asli dengan eksponensial:

$$g(x, y) = e^{s(x, y)} \quad (37)$$

Kapan Menggunakan Homomorphic Filtering?

- Foto dengan pencahayaan tidak merata (bayangan satu sisi).
- Citra medis (X-ray, MRI) dengan rentang dinamis tinggi.
- Foto malam hari atau kondisi cahaya rendah.

Hasilnya: iluminasi dinormalisasi, detail tekstur (reflektansi) diperkuat.

7.5 Contoh Kode: Homomorphic Filtering

Listing 3: Implementasi lengkap homomorphic filtering

```

1 import numpy as np
2 from PIL import Image
3 import matplotlib.pyplot as plt
4
5 def homomorphic_filter(img, D0=30, gamma_L=0.5, gamma_H=2.0, c=1.0):

```

```

6      """
7      Homomorphic filtering untuk penyeimbangan iluminasi.
8      img      : numpy array 2D, nilai [0, 255]
9      D0       : cutoff frequency
10     gamma_L  : penekanan frekuensi rendah (< 1)
11     gamma_H  : penguatan frekuensi tinggi (> 1)
12     c        : kecuraman transisi filter
13     """
14     M, N = img.shape
15
16     # Langkah 1: Ln -- tambah 1 untuk menghindari log(0)
17     ln_img = np.log1p(img.astype(np.float64))
18
19     # Langkah 2: Centering + DFT
20     x, y = np.arange(M), np.arange(N)
21     xx, yy = np.meshgrid(x, y, indexing='ij')
22     center_mask = (-1.0) ** (xx + yy)
23     F = np.fft.fft2(ln_img * center_mask)
24
25     # Langkah 3: Buat filter H(u,v) -- Persamaan homo_filter_func
26     u = np.fft.fftfreq(M) * M
27     v = np.fft.fftfreq(N) * N
28     uu, vv = np.meshgrid(u, v, indexing='ij')
29     D2 = uu**2 + vv**2
30     H = (gamma_H - gamma_L) * (1 - np.exp(-c * D2 / (D0**2))) +
31         gamma_L
32
33     # Langkah 4: Filter + IDFT
34     S = H * F
35     s = np.real(np.fft.ifft2(S)) * center_mask
36
37     # Langkah 5: Eksponensial -- kebalikan log1p
38     g = np.expml(s)
39     return np.clip(g, 0, 255)
40
41 img = np.array(Image.open("image.jpg").convert("L"), dtype=np.float64)
42
43 result = homomorphic_filter(img, D0=30, gamma_L=0.5, gamma_H=2.0, c
44                             =1.0)
45
46 fig, axes = plt.subplots(1, 2, figsize=(10, 5))
47 axes[0].imshow(img.astype(np.uint8), cmap='gray')
48 axes[0].set_title('Citra Asli')
49 axes[1].imshow(result.astype(np.uint8), cmap='gray')
50 axes[1].set_title('Setelah Homomorphic Filtering')
51 for ax in axes:
52     ax.axis('off')
53 plt.tight_layout()
54 plt.show()

```

8 Ringkasan Perbandingan Filter Frekuensi

Filter	Tipe	Tujuan	Ringing	Catatan
ILPF	Lowpass	Smoothing	Parah	Cutoff sempurna
BLPF	Lowpass	Smoothing	Sedang	Dikontrol orde n
GLPF	Lowpass	Smoothing	Tidak ada	Direkomendasikan
IHPF	Highpass	Sharpening	Parah	—
BHPF	Highpass	Sharpening	Sedang	—
GHPF	Highpass	Sharpening	Tidak ada	Direkomendasikan
HFE	Highpass	Sharpening	Sedang	Pertahankan tonalitas
Homomorphic	Campuran	Normalisasi iluminasi	Minimal	Perlu preprocessing log

9 Mini-Latihan (Active Learning)

Latihan 1. Perhitungan Padding Minimal

Jika Anda memiliki citra berukuran 512×512 dan ingin melakukan pemfilteran di domain frekuensi, berapa nilai minimal P dan Q berdasarkan Persamaan (18) agar terhindar dari *wraparound error*? Mengapa dalam praktiknya kita biasanya membulatkan ke pangkat 2 terdekat?

Petunjuk: Hitung $P \geq 2(512) - 1 = 1023$. Pangkat 2 terdekat di atasnya adalah 1024. Bagaimana kaitannya dengan efisiensi algoritma FFT Cooley-Tukey?

Latihan 2. Mengapa GLPF Bebas Ringing?

Jelaskan secara matematis mengapa GLPF (Persamaan (27)) tidak menghasilkan efek *ringing*, sementara ILPF (Persamaan (25)) menghasilkannya. Gunakan Teorema Konvolusi (Persamaan (17)) dalam penjelasan Anda.

Latihan 3. Eksperimen Cutoff Frequency

Dengan menggunakan kode di Bagian 6.5, coba nilai $D_0 \in \{10, 30, 80, 150\}$ untuk GLPF. Dokumentasikan bagaimana D_0 memengaruhi: (a) tingkat kekaburan citra, (b) noise yang tersisa, dan (c) ketajaman tepi.

Latihan 4. Fase vs. Amplitudo

Rekonstruksi sebuah citra menggunakan (a) hanya informasi fase dan (b) hanya amplitudo. Amati hasilnya dan jelaskan mengapa fase lebih penting dalam merepresentasikan struktur visual.

Kode Bantuan:

```
F = np.fft.fft2(img)
# (a) hanya fase: set amplitudo = 1
phase_only = np.real(np.fft.ifft2(np.exp(1j * np.angle(F))
))
```

```
# (b) hanya amplitudo: set fase = 0  
mag_only = np.real(np.fft.ifft2(np.abs(F)))
```

Latihan 5. Aplikasi Medis: GLPF vs. ILPF

Jelaskan mengapa dalam aplikasi medis (citra MRI atau mammogram), GLPF lebih disukai daripada ILPF untuk *smoothing*. Hubungkan jawaban Anda dengan fenomena *ringing* dan risiko artefak yang bisa disalahartikan sebagai kelainan medis.